

```
def update_motivation_state(
    state: MotivationState,
    text: str,
    context_satisfaction: Dict[str, float] = None
) -> MotivationState:
    """Обновляет состояние мотивации на
    основе текста и контекста"""
    intensities = detect_needs_from_text(text)

    # Обновляем интенсивности
    потребностей
    for need_name, intensity in
    intensities.items():
        if need_name in state.needs:
            state.needs[need_name].intensity =
max(
            state.needs[need_name].intensity,
            intensity
        )

    # Учитываем удовлетворенность из
    контекста
```

```
if context_satisfaction:
    for need_name, satisfaction in
context_satisfaction.items():
        if need_name in state.needs:

state.needs[need_name].satisfaction =
satisfaction
```

```
# Вычисляем срочность (intensity -
satisfaction)
urgency_scores = {}
for need_name, need in state.needs.items():
    urgency = need.intensity -
need.satisfaction
    urgency_scores[need_name] = max(0,
urgency)
```

```
# Находим доминирующую потребность
if urgency_scores:
    dominant_need = max(urgency_scores,
key=urgency_scores.get)
    state.dominant_need = dominant_need
```

```
state.urgency =  
urgency_scores[dominant_need]
```

```
# Определяем класс активности  
state.activity_focus =  
state.needs[dominant_need].activity_class
```

```
return state
```